ARCH TECHNOLOGIES

Cyber Security Internship Report

MONTH 1

Full Name	Humza Ahmed
Domain	Cyber Security
Email	humzaahmed798@gmail.com
Phone	+923183366078
Organization	Arch Technologies
Submission	27th April 2026

Table of Contents

1. Introduction to Cyber Security 3
2. Understanding Network Traffic & Packets 4
3. Task 1: Network Traffic Analyzer 5
4. Understanding Keystroke Logging 8
5. Task 2: Keystroke Logger Simulation 9
6. Security Risks & Countermeasures 12
7. Tools & Technologies Used 13
8. Learning Outcomes 14
9. Conclusion 15

1. Introduction to Cyber Security

Cyber Security refers to the discipline of safeguarding systems, networks, applications, and data against unauthorized access, digital threats, and destructive attacks. In an era where virtually every organization depends on interconnected technology, threats range from targeted phishing campaigns to sophisticated nation-state intrusions. This report documents the Month 1 learning journey at Arch Technologies, covering both theoretical frameworks and hands-on practical exercises.

Core Pillars of Cyber Security The CIA Triad

Pillar	Definition	Example
Confidentiality	Restricting data access to authorized individuals only	Encrypted databases, access controls
Integrity	Ensuring information remains accurate and unmodified	Checksums, digital signatures
Availability	Guaranteeing systems remain accessible when required	Redundancy, DDoS protection

Key Domains Explored in This Report

- Network Security: Protecting data during transmission across computer networks
- Ethical Hacking: Authorized vulnerability assessment before malicious actors exploit weaknesses
- Threat Intelligence: Collecting and evaluating data on active cyber threats
- Endpoint Security: Securing individual devices such as laptops and mobile phones
- Cryptography: Applying mathematical techniques to encrypt and protect information

2. Understanding Network Traffic & Packets

Before constructing any network tool, it is essential to understand how data travels across a network. Every internet communication whether a webpage, an email, or a video stream is broken down into discrete units known as packets. Analyzing these packets on an authorized network is a foundational skill for any security professional.

The OSI Model (7 Layers)

Layer	Name	Protocol Examples	Function
7	Application	HTTP, FTP, DNS	User-facing services
6	Presentation	SSL/TLS, JPEG	Data formatting & encryption
5	Session	NetBIOS, RPC	Session management
4	Transport	TCP, UDP	End-to-end delivery
3	Network	IP, ICMP, OSPF	Routing between networks
2	Data Link	Ethernet, Wi-Fi MAC	Node-to-node delivery
1	Physical	Cables, Fiber, RF	Raw bit transmission

Anatomy of a Network Packet

Each packet contains a header (holding source/destination addresses, protocol type, and sequence number) and a payload (the actual transmitted data). Headers allow routers to forward packets correctly, while encryption protects sensitive payload content.

- Source IP / Destination IP : Identifies origin and destination of the packet
- Source Port / Destination Port : Identifies the application (e.g., port 443 = HTTPS)
- Protocol : TCP (reliable, ordered) vs UDP (fast, connectionless)
- TTL (Time To Live) : Prevents packets from looping endlessly through routers
- Checksum : Detects data corruption during transmission
- Payload : The actual transmitted data, often protected by encryption

Note: Capturing network traffic without authorization is a legal offence. All exercises in this report were conducted exclusively on localhost (127.0.0.1) or within a controlled virtual environment with no real user data involved.

3. Task 1: Network Traffic Analyzer

This task demonstrates how to monitor and analyze network packets using Python's built-in socket library on the local loopback interface. The tool listens exclusively to traffic on localhost (127.0.0.1) and does not intercept any external or third-party traffic.

Objectives

- Understand the structure and content of network packet headers
- Visualize how TCP and UDP protocols differentiate various types of traffic
- Parse raw network data using Python
- Grasp why encryption via TLS/SSL is critical for secure communications

Tools & Libraries

Tool / Library	Purpose	Version
Python 3	Core programming language for all scripting	3.10+
socket	Built-in library for raw network socket creation	Built-in
struct	Binary data unpacking for IP/TCP/UDP headers	Built-in
scapy (alt.)	Advanced packet crafting and analysis	2.5+
Wireshark	GUI-based professional packet analyzer	4.x

Implementation Python Network Analyzer

The following script creates a raw socket that listens on the loopback interface and displays real-time packet information including source/destination IPs, protocol type, and port numbers.

```python
import socket, struct, datetime

# Parse IP Header (20 bytes)
def parse_ip_header(data):
iph = struct.unpack("!BBHHHBBH4s4s", data[:20])
return iph[6], socket.inet_ntoa(iph[8]), socket.inet_ntoa(iph[9])

# Parse TCP / UDP
def parse_tcp(data): return struct.unpack("!HHLLBBHHH", data[:20])[:2]
def parse_udp(data): return struct.unpack("!HHHH", data[:8])[:2]
```

```python
# Main Sniffer — Localhost only, 20 packets max
def start_sniffer():
s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
s.bind(("127.0.0.1", 0))
s.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
print("[*] Sniffer started on 127.0.0.1")
count = 0
while count < 20:
raw, _ = s.recvfrom(65535)
ts = datetime.datetime.now().strftime("%H:%M:%S")
proto, src, dst = parse_ip_header(raw)
transport = raw[(raw[0] & 0xF) * 4:]
if proto == 6: sp,dp = parse_tcp(transport); label="TCP "
elif proto == 17: sp,dp = parse_udp(transport); label="UDP "
else: sp=dp=0; label="ICMP"
print(f"[{ts}] {label} {src}:{sp} -> {dst}:{dp}")
count += 1
s.close()

if __name__ == "__main__": start_sniffer()
```

Sample Output

```
[*] Sniffer started on 127.0.0.1
-----------------------------------------------------
[14:23:01] TCP 127.0.0.1:52341 → 127.0.0.1:80
[14:23:01] TCP 127.0.0.1:80 → 127.0.0.1:52341
[14:23:02] UDP 127.0.0.1:53 → 127.0.0.1:53421
[14:23:03] TCP 127.0.0.1:52342 → 127.0.0.1:443
[14:23:03] TCP 127.0.0.1:443 → 127.0.0.1:52342
[14:23:04] ICMP 127.0.0.1 → 127.0.0.1
... (20 packets captured, session complete)
```

Analysis of Output

The captured output reveals important insights. Port 80 traffic indicates unencrypted HTTP communication, while port 443 represents secure HTTPS/TLS connections. UDP port 53 corresponds to DNS lookup activity. The bidirectional TCP entries illustrate the fundamental request-response nature of TCP communication every outbound request generates a corresponding inbound reply.

Wireshark Professional Packet Analyzer

Wireshark is the industry-standard open-source network protocol analyzer used by security teams globally. Compared to the Python script above, Wireshark offers a full graphical interface with color-coded protocol identification, deep packet inspection, and rich statistical analysis capabilities.

No.	Time	Source	Destination	Protocol	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	52341 80 [SYN]
2	0.000123	127.0.0.1	127.0.0.1	TCP	80 52341 [SYN, ACK]
3	0.000245	127.0.0.1	127.0.0.1	TCP	52341 80 [ACK]
4	0.001100	127.0.0.1	127.0.0.1	UDP	53 53421 DNS query
5	0.003200	127.0.0.1	127.0.0.1	ICMP	Echo (ping) request

Figure 1: Wireshark-style packet capture on localhost loopback interface

4. Understanding Keystroke Logging

A keylogger is software or hardware designed to record every keystroke entered on a keyboard. Developing an understanding of keylogger mechanics is a vital component of cyber security education, as they represent one of the most widely deployed attack vectors used by cybercriminals to harvest passwords, credit card details, and private communications.

Types of Keyloggers

Type	How It Works	Detection Difficulty
Software (API-	Hooks into the OS keyboard event API to	Medium

based)	intercept input	Very Hard
Hardware	Physical device inserted between keyboard and PC	Hard (physical inspection)
Browser-based	JavaScript code capturing form input fields	Easy (browser dev tools)
Acoustic	Records keyboard sounds and infers keystrokes from audio patterns	Very Hard

How Attackers Deploy Keyloggers

- Phishing Emails : Malicious attachments silently install keyloggers when opened
- Drive-by Downloads : Visiting compromised websites can trigger automatic malware installation
- Trojan Horses : Legitimate-appearing software bundled with a hidden keylogger payload
- Physical Access : Hardware keylogger inserted into a USB or PS/2 port
- Exploit Kits : Automated tools leveraging browser or operating system vulnerabilities

Legal Warning: Installing a keylogger on any device without the explicit written consent of the owner constitutes a criminal offence in virtually every jurisdiction, including Pakistan under the Prevention of Electronic Crimes Act 2016 (PECA). This exercise was conducted strictly as a controlled simulation on a personal machine.

5. Task 2 : Keystroke Logger Simulation

This simulation leverages Python's pynput library to capture keystrokes typed within a dedicated test terminal window on a personal machine. All captured data is stored in a local log file only no data is transmitted externally. The purpose is to understand attacker methodology and, more critically, to learn how defenders detect and neutralize such tools.

Setup & Requirements

```
# Install required library
pip install pynput

# Run the simulation (test terminal only)
python keylogger_sim.py

# Log file will be created at:
keylog_output.txt (same directory)
```

Implementation Keystroke Logger Simulation

```python
from pynput import keyboard
import datetime
import os

LOG_FILE = "keylog_output.txt"
MAX_KEYS = 200 # Safety limit — stops after 200 keys
key_count = 0

def write_log(text):
with open(LOG_FILE, "a", encoding="utf-8") as f:
f.write(text)

def on_press(key):
global key_count
ts = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
try:
write_log(f"[{ts}] KEY: {key.char}\n")
except AttributeError:
# Special keys (Shift, Ctrl, Enter, etc.)
write_log(f"[{ts}] SPECIAL: {key}\n")
key_count += 1
if key_count >= MAX_KEYS:
print("[*] Max keys captured. Stopping.")
return False # Stops listener

def on_release(key):
if key == keyboard.Key.esc:
print("[*] ESC pressed — logging stopped.")
return False # ESC exits cleanly

# ■■■ Main ■■■
```

```
[SIMULATION] Keystroke Logger Active
[INFO] Press ESC to stop. Max 200 keystrokes.
[INFO] All keys logged to: keylog_output.txt
--------------------------------------------------
[*] Listener started successfully.
... (typing in test window) ...
[*] ESC pressed — logging stopped.
[*] Log saved to C:\Users\Zaid\keylog_output.txt
```

Sample Log Output (keylog_output.txt)

```
=== Session: 2025-04-26 14:30:00 ===
[2025-04-26 14:30:05] KEY: h
[2025-04-26 14:30:05] KEY: e
[2025-04-26 14:30:05] KEY: l
[2025-04-26 14:30:05] KEY: l
[2025-04-26 14:30:05] KEY: o
[2025-04-26 14:30:06] SPECIAL: Key.space
[2025-04-26 14:30:06] KEY: w
[2025-04-26 14:30:06] KEY: o
[2025-04-26 14:30:06] KEY: r
[2025-04-26 14:30:06] KEY: l
[2025-04-26 14:30:06] KEY: d
[2025-04-26 14:30:07] SPECIAL: Key.enter
... 188 more keystrokes ...
```

Figure 2: Log file showing timestamped keystroke records from simulation

6. Security Risks & Countermeasures

Understanding the real-world risks these tools represent and the defensive measures security teams deploy is the ultimate objective of these educational exercises.

Network Sniffing Risks

Risk	Impact	Countermeasure
Credential Theft	Attacker captures login credentials over unencrypted HTTP	Enforce HTTPS/TLS across all web traffic
Session Hijacking	Attacker steals session cookies to impersonate a legitimate user	Implement HttpOnly, Secure, and SameSite cookie flags
Data Exfiltration	Sensitive business data intercepted while in transit	Deploy VPN and end-to-end encryption
ARP Poisoning	Attacker redirects network traffic through their own machine	Enable Dynamic ARP Inspection (DAI) on managed switches

Keylogger Detection & Prevention Strategies

- Antivirus / EDR Software : Solutions such as Windows Defender, Kaspersky, and CrowdStrike detect and quarantine known keylogger signatures
- Virtual Keyboards : On-screen keyboards bypass software-based keystroke interception
- Two-Factor Authentication (2FA) : Renders captured passwords useless without the second authentication factor
- Password Managers : Auto-fill functionality eliminates keystroke exposure during password entry
- Behavioral Analysis : EDR platforms flag unusual process activity indicative of keyloggers
- Regular Security Audits : Scanning for unauthorized or suspicious processes running on endpoints
- User Education : Training personnel to recognize phishing and avoid suspicious downloads

7. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.10+	Core language for all scripting, automation, and implementation
pynput Library	Cross-platform keyboard and mouse listener for keystroke simulation
socket Module	Raw socket creation for network traffic capture and analysis
struct Module	Binary data unpacking for IP, TCP, and UDP protocol headers
Wireshark 4.x	Professional GUI-based packet analyzer (open source)
Windows Defender	Endpoint protection and keylogger detection testing
VS Code	Code editor with integrated Python debugging support
Git	Version control for all project code and documentation
VirtualBox	Safe, isolated virtual environment for security exercises

8. Learning Outcomes

Upon completing Month 1 tasks, the following skills and knowledge areas were developed and consolidated:

Network Fundamentals

- Understood all seven OSI model layers and their specific roles in data transmission
- Learned the internal structure of IP packets including header fields and payload
- Differentiated between TCP (reliable, connection-oriented) and UDP (fast, connectionless) protocols

- Mapped common port numbers to their associated services and applications

Python Programming Skills

- Utilized raw sockets to access and analyze network layer data
- Applied struct.unpack() to parse binary protocol headers accurately
- Implemented event-driven programming patterns using pynput listeners
- Practiced file I/O operations for data logging and persistence

Security Awareness

- Understood the mechanisms by which attackers exploit packet sniffers to steal credentials
- Recognized why HTTPS/TLS encryption is non-negotiable for all web applications
- Identified the primary vectors through which keyloggers are deployed and spread
- Learned practical defensive strategies including 2FA, EDR solutions, VPNs, and encryption

9. Conclusion

This Month 1 Cyber Security Internship Report at Arch Technologies has established a strong foundational understanding of two pivotal security topics: network traffic analysis and keystroke logging. Both subjects were approached exclusively from a defender's perspective comprehending how these attack techniques function is the essential first step toward constructing robust, effective defenses.

The network analyzer exercise demonstrated how data traverses modern networks, illustrated the dangers of unencrypted protocols, and showcased how tools like Wireshark empower security teams to monitor their own infrastructure proactively. The keystroke logger simulation underscored how easily credentials can be compromised on inadequately protected systems and reinforced the indispensable role of endpoint protection solutions such as Windows Defender and multi-factor authentication.

Looking ahead to Month 2, the planned focus includes vulnerability scanning using Nmap, foundational penetration testing methodology, and web application security concepts drawn from the OWASP Top 10. The knowledge and practical experience acquired during Month 1 have laid a solid and reliable foundation for deeper security research and more advanced challenges.

Humza Ahmed
Cyber Security Intern | Month 1

Email: humzaahmed798@gmail.com | Phone: +923183366078
Organization: Arch Technologies | Submission: 27th April 2026